

LogiFlow on GCP

Layman → every Cloud Run knob → full request & deploy path

team-3mo

2 CPU · 2 GiB

min 1 · max 3

asia-south1

Production Cloud Run — memorize first

2 vCPU

CPU

2 GiB

MEMORY

min 1

ALWAYS WARM

max 3

COST CEILING

40

CONCURRENCY

300 s

TIMEOUT

Service logiflow-api · **Region** asia-south1 · **Profile** team-3mo · **Workers**

COMPOSE_PARALLEL_WORKERS=8 · **Warmup** /api/warm-backend (load, focus, every 3 min)

One line: FastAPI on Cloud Run at 2 CPU / 2 GiB, min 1 max 3, concurrency 40, 300s timeout — sized after Render's **512 MB** OOM'd on compose and rail CSVs.

How to read this doc

Part B = layman. Part C = every knob with LogiFlow's real value. Part D–E = live request + deploy. Part F = interview answers.

Part B — Layman start

What problem are we solving?

LAYMAN LogiFlow's brain is a Python FastAPI app. Judges and users need it online with enough RAM for heavy compose jobs. GCP Cloud Run is the rented computer that runs our Docker image and gives an HTTPS URL.

Two boxes that matter

1. **Frontend (Next.js)** — on **Vercel**. Browser → Vercel → API proxy.
2. **Backend (FastAPI)** — on **GCP Cloud Run** in `asia-south1`. This PDF is about that box.

LAYMAN A Docker image is a sealed lunchbox: Python 3.11 + code + libraries. Cloud Run opens it and runs `uvicorn` on port 8080. Same lunchbox locally and in prod.

Cloud Run is not the database. Postgres / Supabase / Redis live elsewhere. Cloud Run is stateless compute — instances can die; durable state stays outside.

Part C — Every Cloud Run knob (LogiFlow values)

C1. Memory — why 2 GiB?

LAYMAN RAM is working space for one container. Need more than the limit → platform kills the process = **OOM**.

Host	RAM	What happened
Render free (old)	512 MB	Rail CSV / compose → OOM crashes
Cloud Run free	1 GiB	Student budget; still tight for heavy compose
Cloud Run team-3mo (prod)	2 GiB	Headroom for workers + model + in-process caches

Why LogiFlow: Compose runs modes/legs in parallel; rail data + sklearn + geometry need RAM. Also set `RAIL_PRELOAD_ON_STARTUP=false` so huge CSVs are not loaded at boot.

Q. Why 2 GiB and not 512 MB or 4 GiB?

YOU SAY

We OOM'd on Render at 512 MB. 2 GiB was enough with 8 compose workers. 4 GiB would cost more without a measured need — we sized up from a real crash.

C2. CPU — why 2 vCPU?

LAYMAN CPU = cores the container can use. Parallel Python work wants more than one.

Why LogiFlow: `COMPOSE_PARALLEL_WORKERS=8` and multi-mode optimize (road/rail/air/water) need 2 vCPU. Free profile uses 1 CPU to save money.

C3. Min instances — why 1 (not 0)?

LAYMAN Min instances = containers kept running when idle. Min 0 = scale to zero (cheap, cold start). Min 1 = always warm (pay for latency).

Profile	Min	User experience
free	0	Idle ≈ \$0; first hit after sleep pays 5–15 s cold start
team-3mo (prod)	1	Always warm — demos / compose start immediately

Why LogiFlow: Solution Challenge demos cannot eat a 10s wake. We buy warmth from the 3-month credit budget. Frontend still hits `/api/warm-backend` on load, focus, and every 3 minutes.

Q. What is the cost of min instances = 1?

YOU SAY

We give up scale-to-zero billing to buy predictable latency. If budget forced us, drop to free profile (min 0, 1 GiB) and lean on warmup pings.

C4. Max instances — why 3?

LAYMAN Hard ceiling on container count — stops spikes/abuse from multiplying bill and DB connections.

Why LogiFlow: Rate limits ~8/min/IP; optimize semaphore ~5. Max 3 caps worst-case spend and connection fan-out.

C5. Concurrency — why 40?

LAYMAN How many HTTP requests one container handles at once. Higher = fewer containers (cheaper); too high = CPU queueing and latency cliffs.

Why LogiFlow: Deploy sets `--concurrency 40`. FastAPI is async for I/O, but scoring/compose burn CPU. Tune from p95 metrics — not folklore.

C6. Timeout — why 300 seconds?

Layer	Budget
Vercel <code>/api/compose</code>	90 s <code>maxDuration</code>
Cloud Run request timeout	300 s
Per-mode optimize futures	~30 s each

Why LogiFlow: Hybrid compose can run 60–90+ seconds. Short host timeouts failed us; 300s matches the product.

C7. Profiles side by side

Knob	team-3mo (prod)	free	Old Render
Min instances	1	0	n/a (sleep)
Max instances	3	lower	1 box
CPU	2	1	shared tiny
Memory	2 GiB	1 GiB	512 MB
Timeout	300 s	300 s	shorter / sleep
Cold start	≈0 (warm)	5–15 s	30–90 s after sleep

Interview rule. If you say “Cloud Run,” immediately name **2 CPU / 2 GiB / min 1 / max 3 / concurrency 40 / 300s**.

Part D — End-to-end live request

Happy path (optimize / rank)

1. Browser loads Next.js on **Vercel**.
2. UI calls same-origin `/api/backend/...` (no CORS pain; hides raw Cloud Run URL).
3. Vercel proxies to `https://logiflow-api-....run.app`.
4. Cloud Run TLS edge terminates HTTPS; container sees HTTP on **8080**.
5. With **min instances = 1**, a warm instance usually exists.

6. FastAPI validates JWT/input; slowapi may rate-limit (~8/min/IP).
7. Pipelines run. Cache-aside: memory → Redis → Supabase.
8. JSON returns; UI renders ranked options / map.

Compose path (why 2 GiB + 300s exist)

1. UI hits Vercel `POST /api/compose` (90s) or stream endpoints.
2. Cloud Run `RouteComposer` fans out legs with **8** parallel workers.
3. Leg cache: L1 memory → L2 Redis (~6h TTL) → L3 Supabase.
4. Redis down → fall back to memory/Supabase/recompute — do not 500 the whole request.

Part E — End-to-end deploy

LAYMAN Deploy = freeze code into an image → store it → tell Cloud Run to run that image with these knobs → keep old revision for rollback.

1. `gcloud builds submit backend` → image `gcr.io/.../logiflow-api`.
2. `team-3mo` ⇒ min 1, max 3, CPU 2, memory **2Gi**.
3. `gcloud run deploy` with `--memory 2Gi --cpu 2 --timeout 300 --concurrency 40 --min-instances 1 --max-instances 3 --port 8080`.
4. Env from `backend/.env` (never committed): Gemini, TomTom, Supabase, JWT, Redis, Google client id...
5. Runtime: `COMPOSE_PARALLEL_WORKERS=8`, `RAIL_PRELOAD_ON_STARTUP=false`.
6. Smoke: `curl .../health` → `{"status":"ok"}`.

Identity split

Deploy SA (`logiflow-deploy`) builds/pushes/deloys. Runtime SA is the running service. User Google Sign-In is a third identity — not the deploy key.

Part F — Interview scripts

Q. Walk me through LogiFlow on GCP.

YOU SAY

Frontend on Vercel; FastAPI on Cloud Run in asia-south1. Production is 2 vCPU, 2 GiB, min 1, max 3, concurrency 40, 300s timeout. We left Render's 512 MB after OOM on compose/rail. Redis and Supabase hold shared cache and durable data; Cloud Run stays stateless.

Q. Explain min / max / memory / CPU for your service.

YOU SAY

Min 1 keeps one warm container for judges. Max 3 caps cost and connections. 2 GiB is the room compose needed after 512 MB failed. 2 CPU matches parallel pipelines. Concurrency 40 multiplexes requests until CPU latency says otherwise.

Q. Why not Kubernetes?

YOU SAY

One primary API container. Cloud Run gave HTTPS, revisions, and the exact CPU/RAM knobs. GKE would add cluster overhead we did not earn.

Q. Redis down?

YOU SAY

Fall back to memory and Supabase, then recompute. Redis speeds repeats across instances — it is not the source of truth for tariffs.

Do not invent: GKE, “Redis is the DB,” “512 MB was fine,” “cold starts don’t matter.”
Do say: Render 512 MB OOM → Cloud Run 2 GiB; min 1 for demos; 300s for compose; Vercel proxies; warmup every 3 min.

Part G — Pocket card

Stack split	Vercel (UI) + Cloud Run (API)
Region / profile	asia-south1 / team-3mo
CPU / RAM	2 vCPU / 2 GiB
Min / Max	1 / 3
Concurrency / timeout	40 / 300 s (Vercel compose 90 s)
Workers / rail boot	8 / preload false
Rate limit / optimize guard	~8/min/IP · semaphore ~5 + 1h cache
Cache tiers	memory → Redis → Supabase
Origin story	Render 512 MB OOM → Cloud Run 2 GiB + min 1